

The background is a solid dark blue with abstract geometric shapes in lighter shades of blue. A large, light blue curved shape is on the right side, and a dark blue diagonal shape is at the bottom right.

Computer Science

03. Process & Thread

3주차: 프로세스와 스레드

오늘의 핵심 질문

- 💡 1. 실행 중인 프로그램은 어디에 저장되고 관리될까?
- 💡 2. 스레드는 왜 만들어졌을까?
- 💡 3. Chrome은 어떻게 여러 탭을 동시에 실행할 수 있을까?

목차 INDEX

01 Process

- 프로세스
- PID
- 프로세스의 자원
- IPC

02 Thread

- 프로세스의 문제점
- 스레드
- TID
- 병렬성
- 동시성

03 Context Switching

- Context Switching
- Context Switching Cost

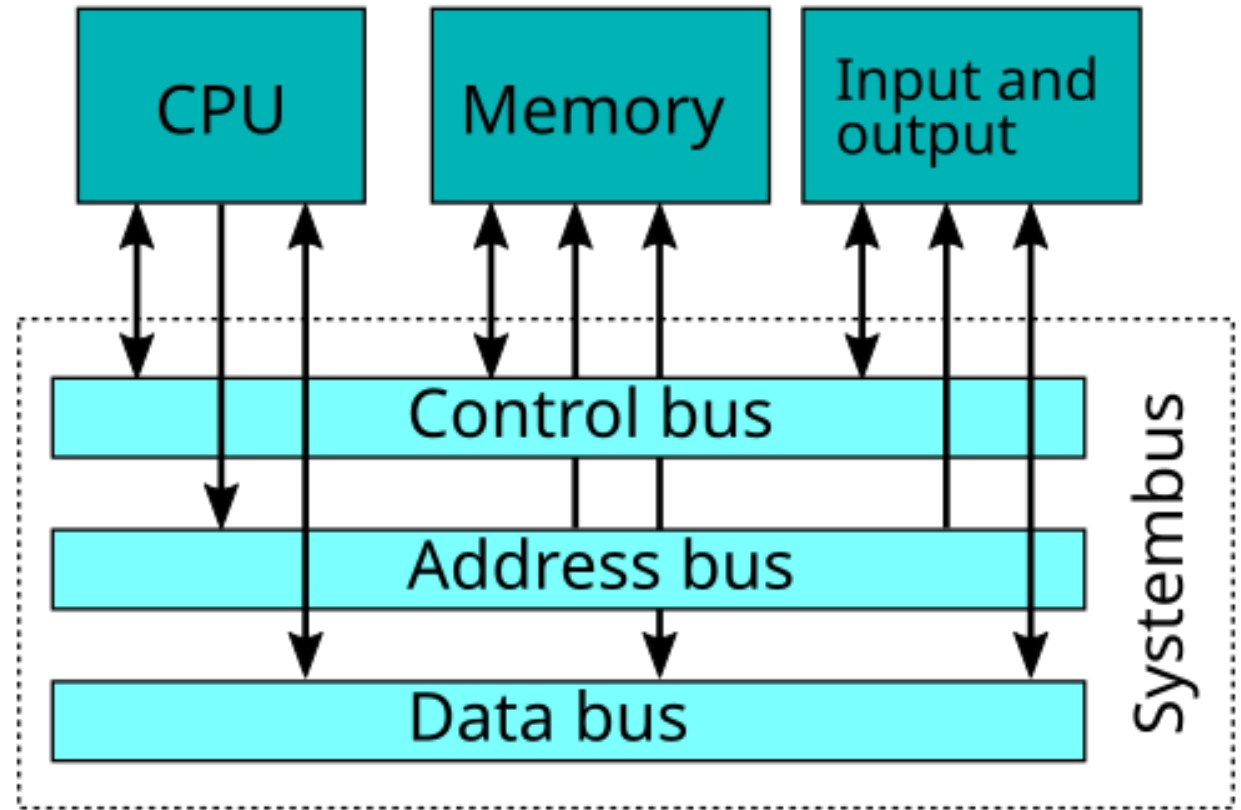
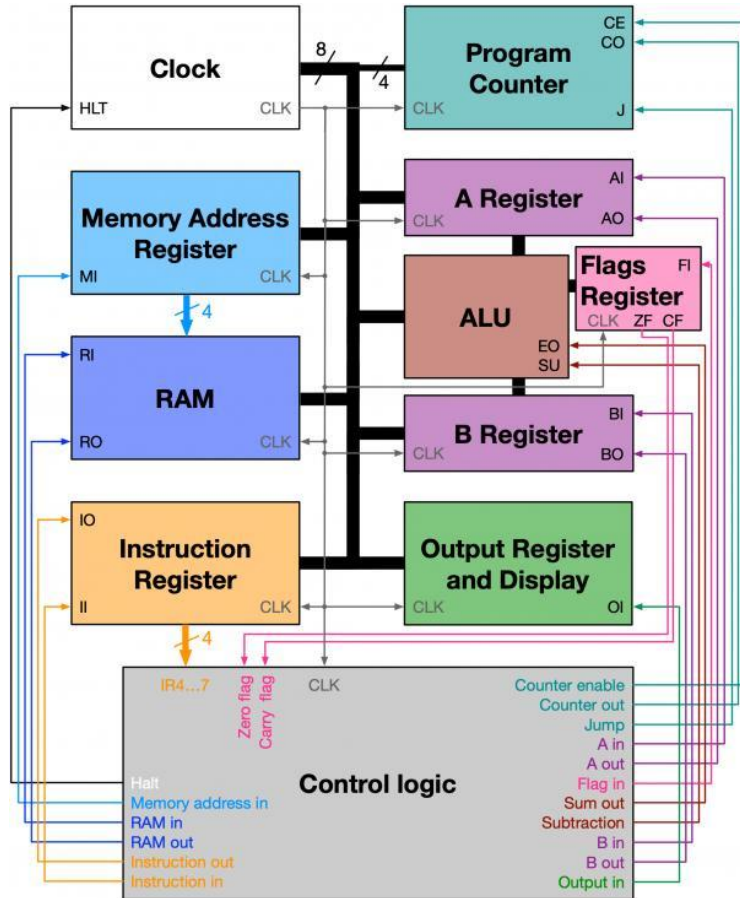
01

Process

프로세스

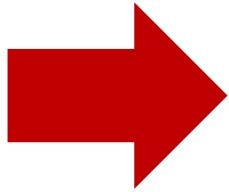
01 Process

1주, 2주차에 배운거



01 Process

1주, 2주차에 배운거



```
01010101
01001000 10001001 11100101
10111111 00000100 00100000 01000000 00000000
11101000 11111100 11111110 11111111 11111111
10111000 00000000 00000000 00000000 00000000
01011101
11000011
```

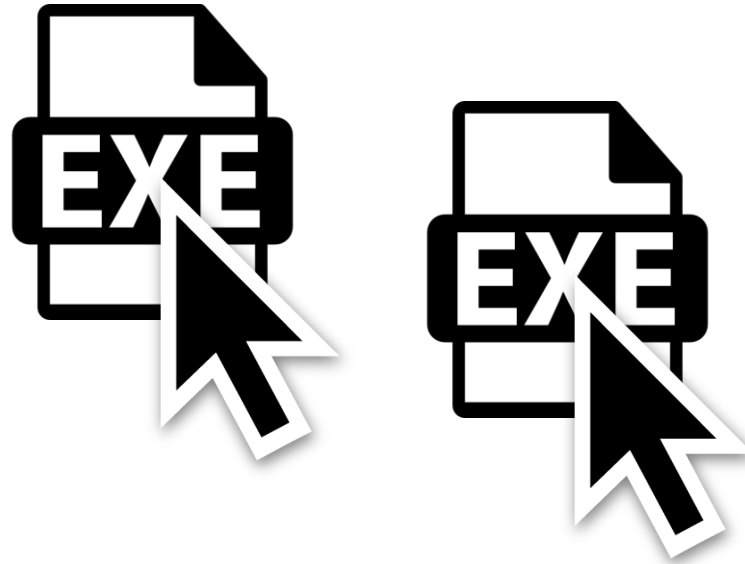
Binary

```
push rbp
mov rbp, rsp
mov edi, 0x402004
call puts
mov eax, 0
pop rbp
ret
```

Assembly

01 Process

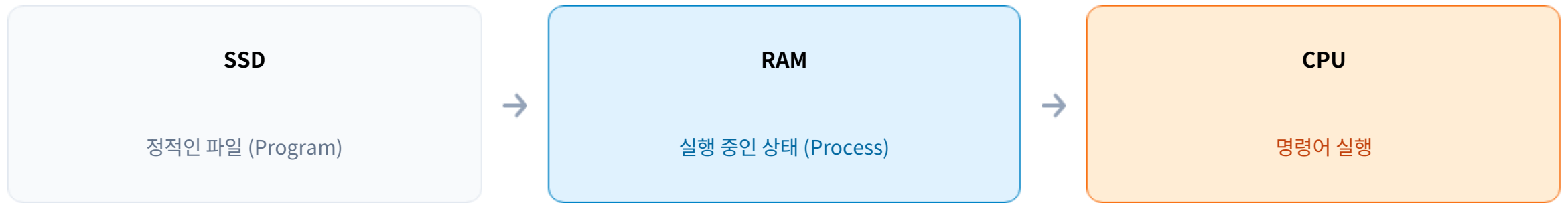
오늘 배울거



2개 동시에 실행시키면 어떡함

01 Process

복습: 프로그램 실행 시 과정



프로세스

```
class 프로그램:  
    pass  
  
프로세스 = 프로그램()
```

프로세스 == 실행 중인 프로그램의 인스턴스

01 Process

프로세스

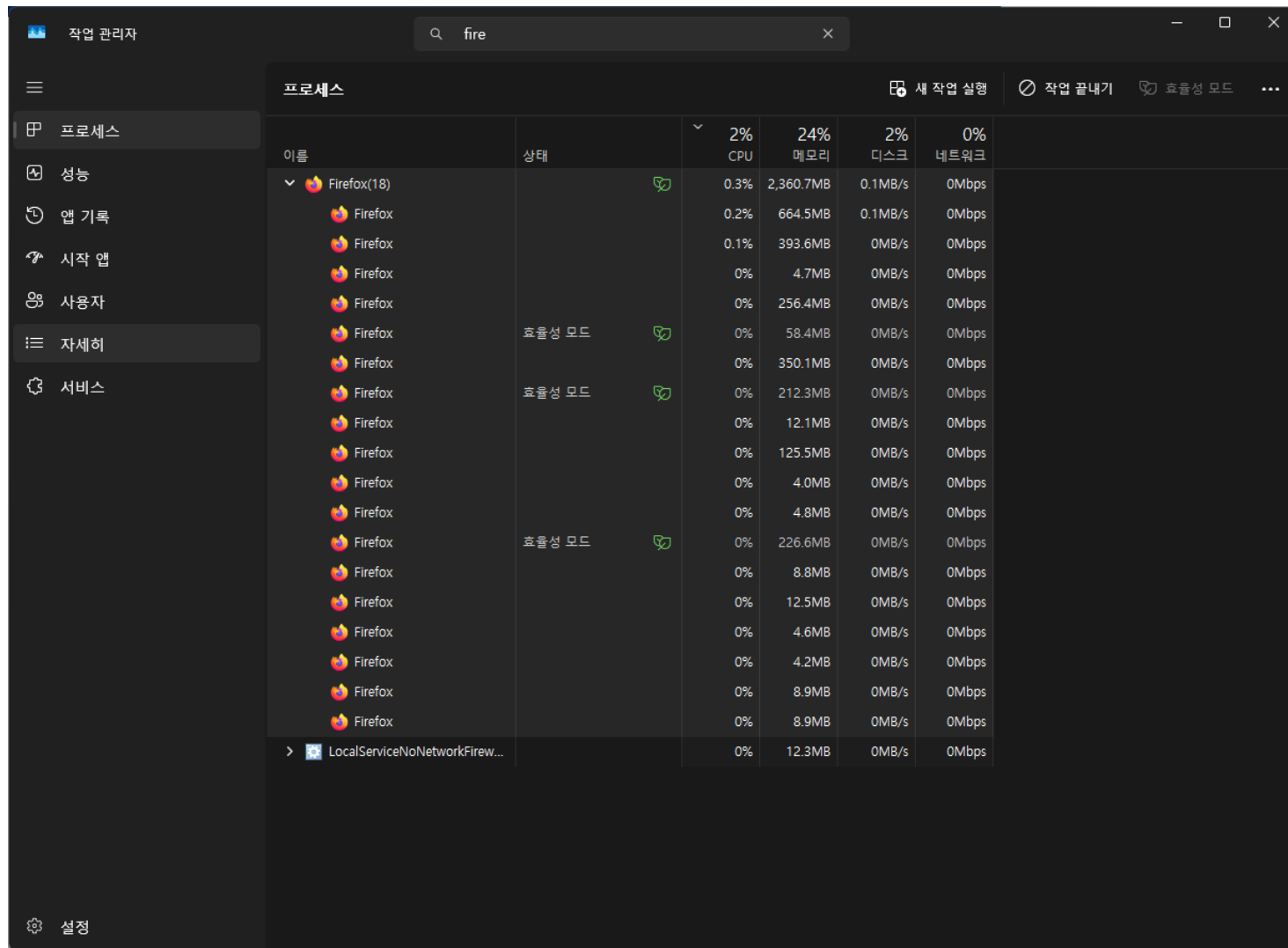
구분	Program (프로그램)	Process (프로세스)
성격	정적 (Static)	동적 (Dynamic)
위치	SSD / HDD 등 저장장치	RAM (메모리)
정의	실행 가능한 코드와 데이터 묶음	운영체제에 의해 실행 중인 프로그램의 인스턴스
특징	파일 형태	PID, 메모리 공간, CPU 상태 등 자원 보유

같은 파이썬 파일을 두 번 실행하면
서로 다른 프로세스 2개 생성

01 Process

프로세스

- 윈도우의 경우 작업관리자에서 확인 가능
- SSD에 저장된 Firefox의 코드가 CPU에서 실행되며 18개의 인스턴스(프로세스) 생성
- 각 프로세스마다 하는 일이 다르고, 자원 소모량도 다름
- 리눅스나 맥에서는 어떻게 보는지 몰?루 (알아서 찾아보셈)



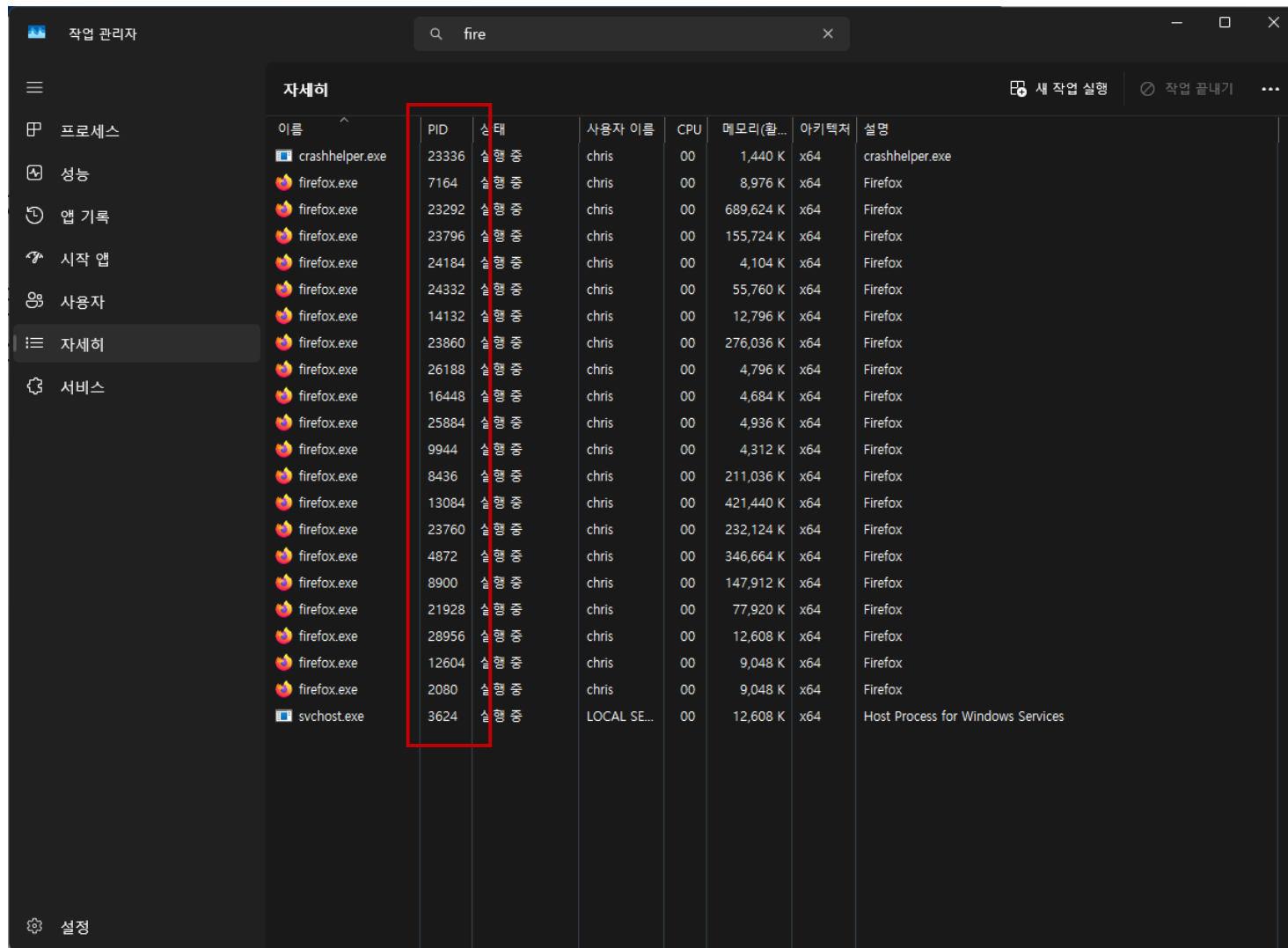
The screenshot shows the Windows Task Manager window with the 'Processes' tab selected. The search bar at the top contains 'fire'. The left sidebar shows the 'Processes' section expanded. The main area displays a list of processes, with 18 instances of Firefox (Firefox(18)) expanded to show individual process details. The columns are: 이름 (Name), 상태 (Status), CPU, 메모리 (Memory), 디스크 (Disk), and 네트워크 (Network). The processes are listed in descending order of memory usage. The first instance is using 0.3% CPU and 2,360.7MB of memory. The last instance is using 0% CPU and 12.3MB of memory.

이름	상태	CPU	메모리	디스크	네트워크
Firefox(18)		2%	24%	2%	0%
Firefox		0.3%	2,360.7MB	0.1MB/s	0Mbps
Firefox		0.2%	664.5MB	0.1MB/s	0Mbps
Firefox		0.1%	393.6MB	0MB/s	0Mbps
Firefox		0%	4.7MB	0MB/s	0Mbps
Firefox		0%	256.4MB	0MB/s	0Mbps
Firefox	효율성 모드	0%	58.4MB	0MB/s	0Mbps
Firefox		0%	350.1MB	0MB/s	0Mbps
Firefox	효율성 모드	0%	212.3MB	0MB/s	0Mbps
Firefox		0%	12.1MB	0MB/s	0Mbps
Firefox		0%	125.5MB	0MB/s	0Mbps
Firefox		0%	4.0MB	0MB/s	0Mbps
Firefox		0%	4.8MB	0MB/s	0Mbps
Firefox	효율성 모드	0%	226.6MB	0MB/s	0Mbps
Firefox		0%	8.8MB	0MB/s	0Mbps
Firefox		0%	12.5MB	0MB/s	0Mbps
Firefox		0%	4.6MB	0MB/s	0Mbps
Firefox		0%	4.2MB	0MB/s	0Mbps
Firefox		0%	8.9MB	0MB/s	0Mbps
Firefox		0%	8.9MB	0MB/s	0Mbps
LocalServiceNoNetworkFirew...		0%	12.3MB	0MB/s	0Mbps

01 Process

PID Process ID

- OS가 여러 프로세스를 구분하기 위해 붙이는 고유한 번호
- 동일한 프로그램을 실행해도 PID는 다름
- OS는 PID로 프로세스를 관리 및 제어함



작업 관리자

자세히

이름	PID	상태	사용자 이름	CPU	메모리(활...	아키텍처	설명
crashhelper.exe	23336	실행 중	chris	00	1,440 K	x64	crashhelper.exe
firefox.exe	7164	실행 중	chris	00	8,976 K	x64	Firefox
firefox.exe	23292	실행 중	chris	00	689,624 K	x64	Firefox
firefox.exe	23796	실행 중	chris	00	155,724 K	x64	Firefox
firefox.exe	24184	실행 중	chris	00	4,104 K	x64	Firefox
firefox.exe	24332	실행 중	chris	00	55,760 K	x64	Firefox
firefox.exe	14132	실행 중	chris	00	12,796 K	x64	Firefox
firefox.exe	23860	실행 중	chris	00	276,036 K	x64	Firefox
firefox.exe	26188	실행 중	chris	00	4,796 K	x64	Firefox
firefox.exe	16448	실행 중	chris	00	4,684 K	x64	Firefox
firefox.exe	25884	실행 중	chris	00	4,936 K	x64	Firefox
firefox.exe	9944	실행 중	chris	00	4,312 K	x64	Firefox
firefox.exe	8436	실행 중	chris	00	211,036 K	x64	Firefox
firefox.exe	13084	실행 중	chris	00	421,440 K	x64	Firefox
firefox.exe	23760	실행 중	chris	00	232,124 K	x64	Firefox
firefox.exe	4872	실행 중	chris	00	346,664 K	x64	Firefox
firefox.exe	8900	실행 중	chris	00	147,912 K	x64	Firefox
firefox.exe	21928	실행 중	chris	00	77,920 K	x64	Firefox
firefox.exe	28956	실행 중	chris	00	12,608 K	x64	Firefox
firefox.exe	12604	실행 중	chris	00	9,048 K	x64	Firefox
firefox.exe	2080	실행 중	chris	00	9,048 K	x64	Firefox
svchost.exe	3624	실행 중	LOCAL SE...	00	12,608 K	x64	Host Process for Windows Services

새 작업 실행 작업 끝내기

설정

프로세스에게 할당되는 것들

실행 코드 및 데이터

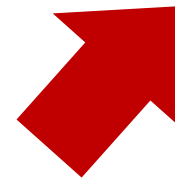
CPU가 실행할 명령어와 전역 변수 등
프로그램 본연의 자원

독립된 메모리 공간

다른 프로세스가 침범할 수 없는
자신만의 가상 주소 공간 (Stack, Heap)

관리 정보 (PCB)

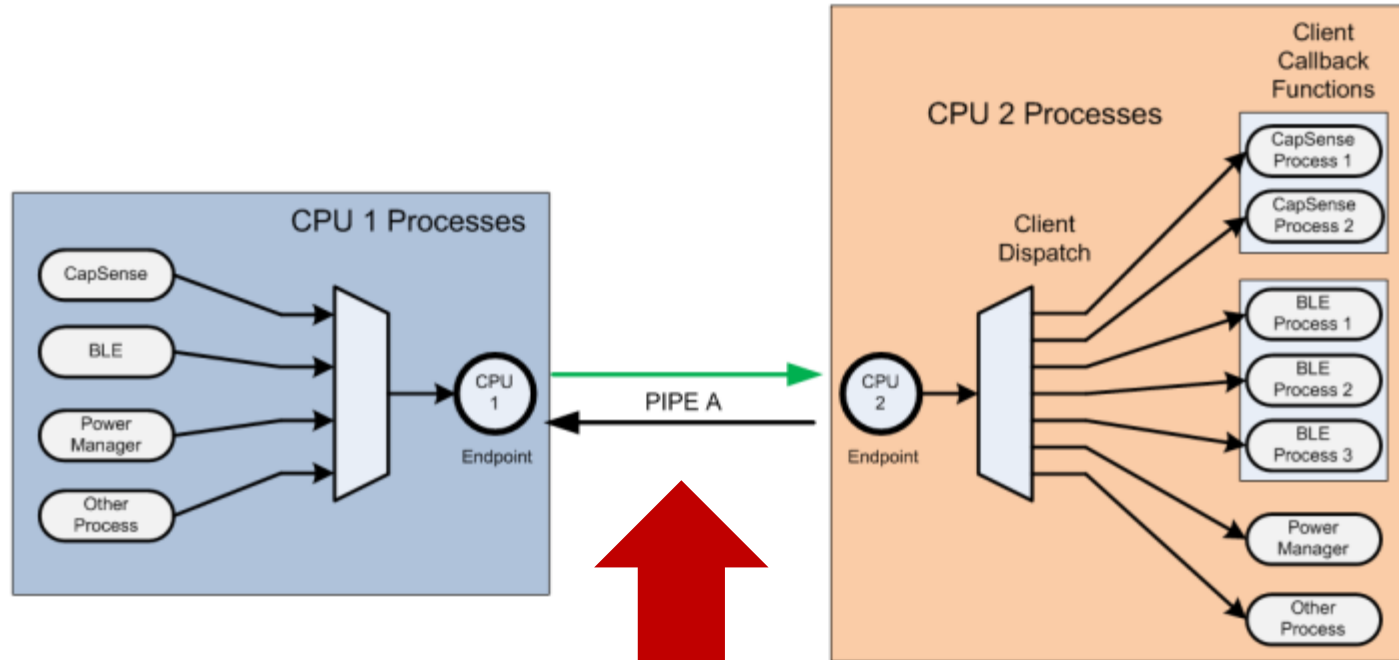
PID, Register 상태, 실행 상태 등을 담은
운영체제의 관리 블록



Process Control Block

01 Process

프로세스 간 데이터 통신: IPC



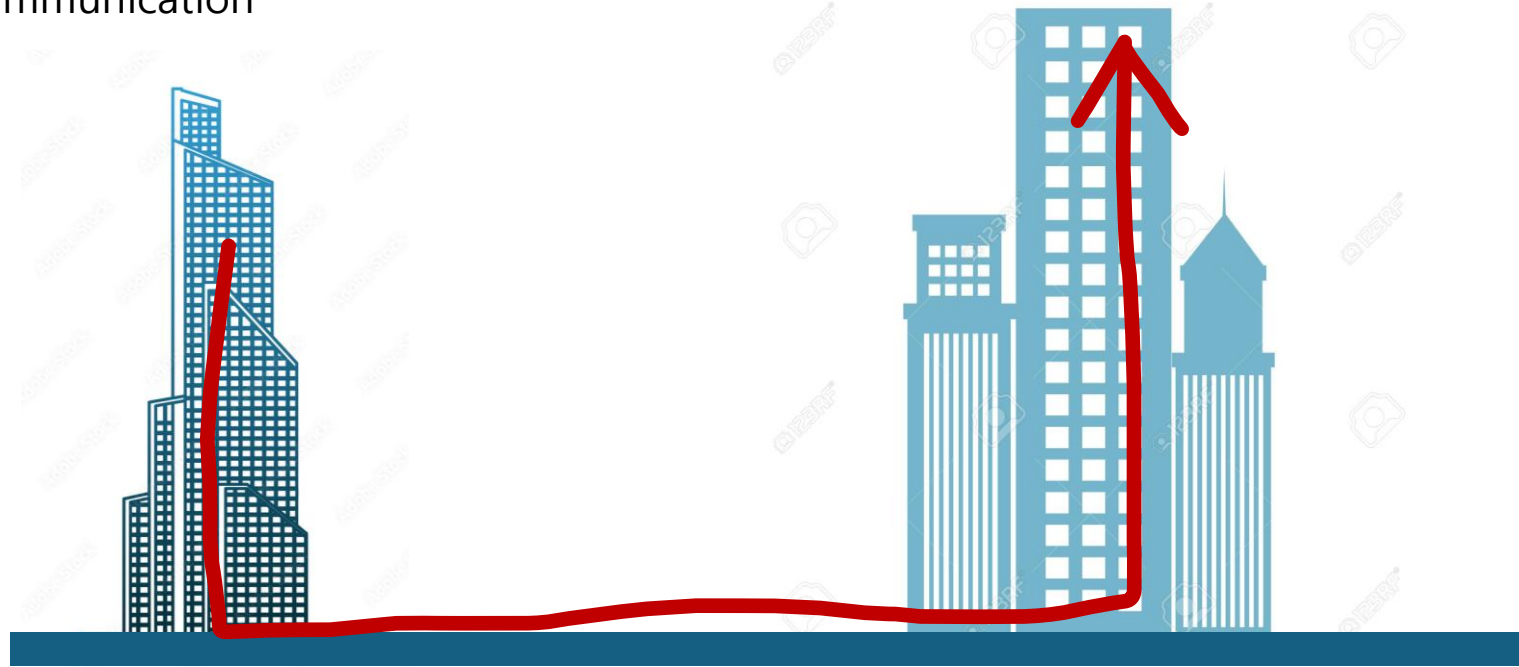
프로세스 A

프로세스 B

프로세스 간 데이터 통신

01 Process

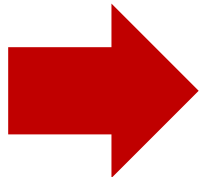
IPC Inter-Process Communication



프로세스 A

프로세스 B

데이터가 프로세스를 이동하려면 프로세스 바깥으로 나가
IPC를 통해 정식으로 이동해야 함

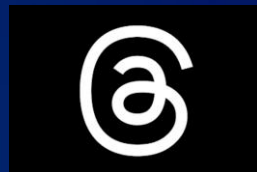


근데 $A \rightarrow B$ 로 가기 좀 불편한듯?

02

스레드

Thread



이거 아님

프로세스만 사용할 때의 문제점

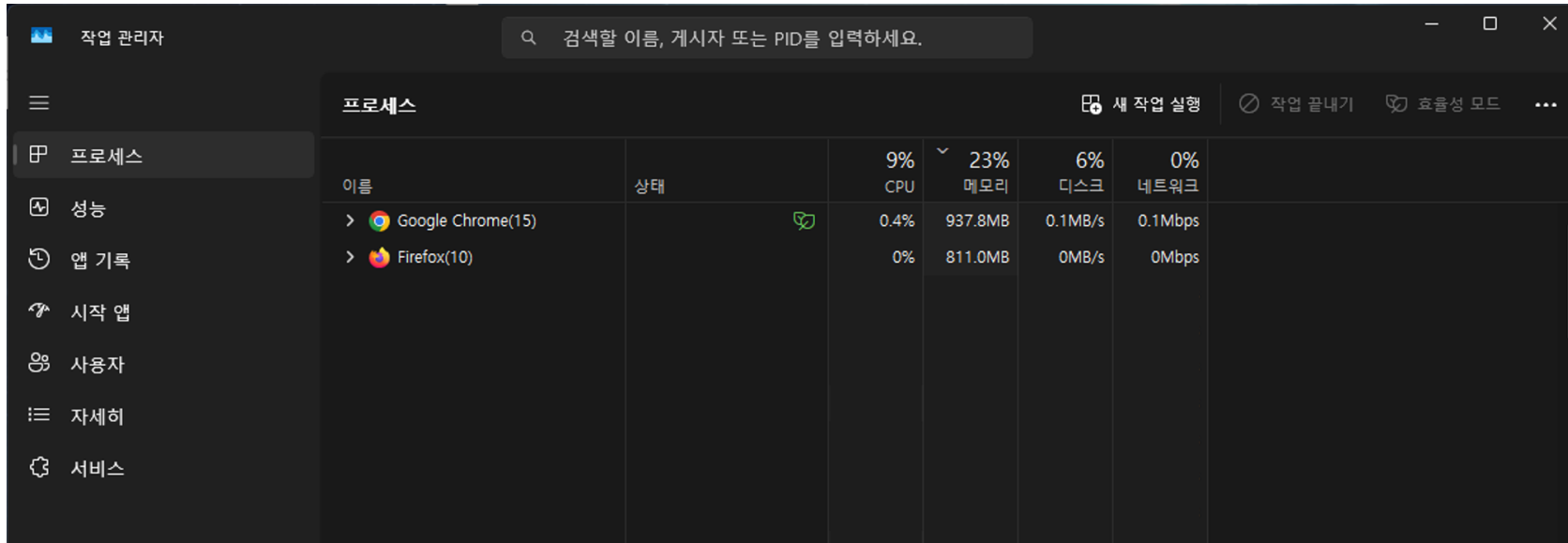
- 프로세스를 하나 만들 때 별도의 메모리를 할당해야 함
- 하나의 작업마다 완전히 별도의 프로세스를 만들면 메모리를 너무 많이 잡아먹게 됨
- 프로세스 간 데이터를 주고받는 IPC 작업이 매우 복잡해지고 번거로워짐
- 크롬은 설계 철학 상 프로세스를 많이 사용함
(그래서 타 브라우저 대비 램 소모량이 평균적으로 많음)

독립된 메모리 공간

다른 프로세스가 침범할 수 없는
자신만의 가상 주소 공간 (Stack, Heap)

프로세스 만들 때 필요한 것

프로세스만 사용할 때의 문제점



작업 관리자						
검색할 이름, 게시자 또는 PID를 입력하세요.						
프로세스						
새 작업 실행 작업 끝내기 효율성 모드						
이름	상태	9% CPU	23% 메모리	6% 디스크	0% 네트워크	
> Google Chrome(15)		0.4%	937.8MB	0.1MB/s	0.1Mbps	
> Firefox(10)		0%	811.0MB	0MB/s	0Mbps	

(컴퓨터 켜자마자 바로 찍음)

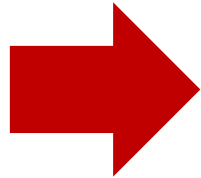
크롬은 프로세스 15개, 파폭은 프로세스 10개만 켜진 것을 볼 수 있음
메모리도 크롬이 100MB 이상 더 차지함

프로세스만 사용할 때의 문제점

상황 가정

하나의 프로그램 안에서 동시에 사용자 입력 처리, 네트워크 요청, 파일 다운로드, 화면 갱신을 해야 합니다.

- 프로세스 1: 사용자 입력 처리
- 프로세스 2: 네트워크 요청
- 프로세스 3: 파일 다운로드
- 프로세스 4: 화면 갱신
- ⋮



공용 메모리 만들면 좀 낫지 않을까?

스레드 Thread

정의: 프로세스 내부에서 실제 실행 흐름을 담당하는 최소 단위

하나의 프로세스는 최소 1개 이상의 스레드(Main Thread)를 가짐

프로세스의 자원(Code, Data, 메모리)을 공유하면서 각자의 실행 흐름을 가짐

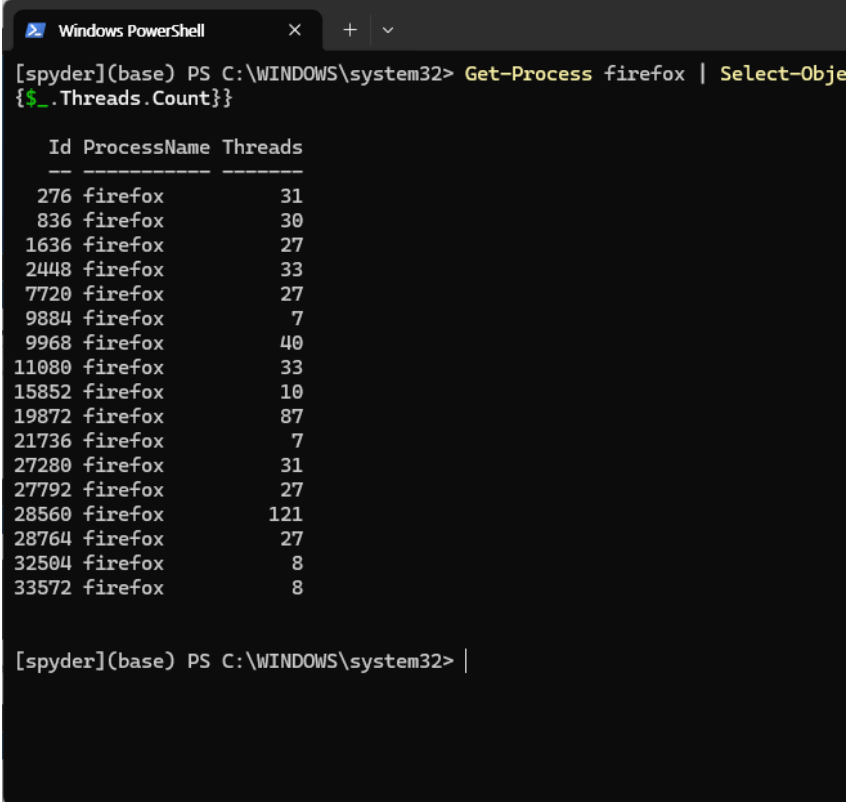
Process & Thread 구조

Process

- └─ Code / Data / Memory (공유)
- └─ **Thread 1** (Stack + Register)
- └─ **Thread 2** (Stack + Register)

스레드 Thread

- 사용자가 파이어폭스를 한 번 실행했을 때 OS가 몰래 프로세스를 17개 생성함
- PID 28650 프로세스는 스레드를 121개, PID 276 프로세스는 스레드를 31개 가진다는 뜻
- 하나의 프로세스는 여러 개의 스레드를 가질 수 있음
- 프로세스와 유사하게 각 스레드마다 ID(TID)를 가짐



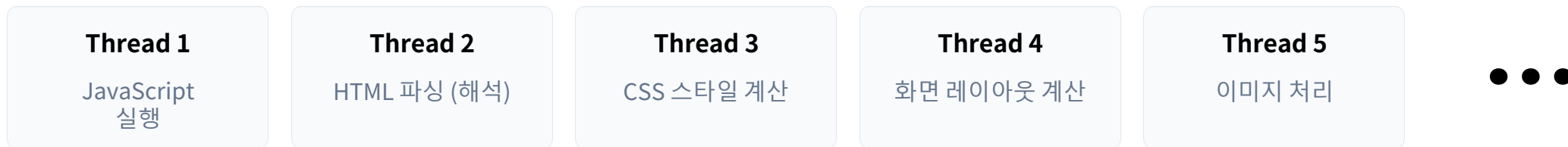
```
[spyder](base) PS C:\WINDOWS\system32> Get-Process firefox | Select-Object $_.Threads.Count
```

Id	ProcessName	Threads
276	firefox	31
836	firefox	30
1636	firefox	27
2448	firefox	33
7720	firefox	27
9884	firefox	7
9968	firefox	40
11080	firefox	33
15852	firefox	10
19872	firefox	87
21736	firefox	7
27280	firefox	31
27792	firefox	27
28560	firefox	121
28764	firefox	27
32504	firefox	8
33572	firefox	8

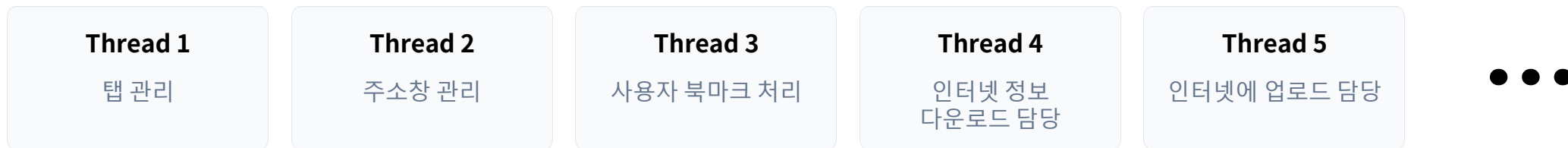
```
[spyder](base) PS C:\WINDOWS\system32> |
```

실제로 파워셸에서 파이어폭스의 스레드 목록을 출력해봄

웹브라우저 프로세스의 구조



Process 1: 웹페이지 출력 담당



Process 2: 내부 기능 담당

스레드가 공유하는 것 · 공유하지 않는 것

공유하는 것 (Shared)

- Code / Text 영역
- Data 영역 (전역 변수 등)
- 프로세스 내부 메모리
- 열린 파일 및 OS 자원

공유하지 않는 것 (Independent)

- 각자 호출한 함수와 지역 변수
- Register 상태 (PC 포함, 각자의 실행 위치)
- Thread ID

프로세스와 스레드 비교

비교 항목	Process (프로세스)	Thread (스레드)
실행 단위	운영체제 자원 할당의 기본 단위	CPU 실행 (개별 작업)의 기본 단위
메모리 공간	완전 독립적 (가상 주소 분리)	코드, 데이터, 공용 메모리 공유
생성/전환 비용	큼 (무거움)	상대적으로 작음 (가벼움)
장애 영향	한 프로세스의 오류가 다른 곳에 영향 없음	한 스레드의 치명적 오류가 전체 프로세스에 영향 가능

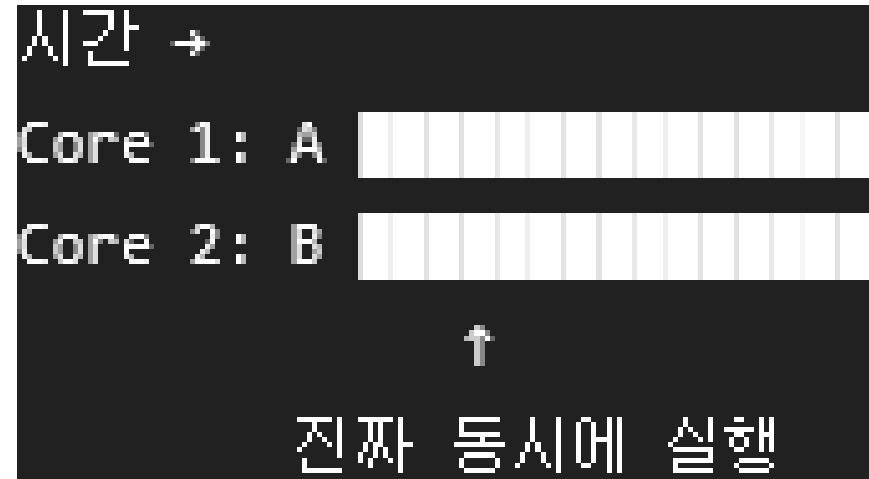
02 스레드 Thread

병렬성 Parallelism

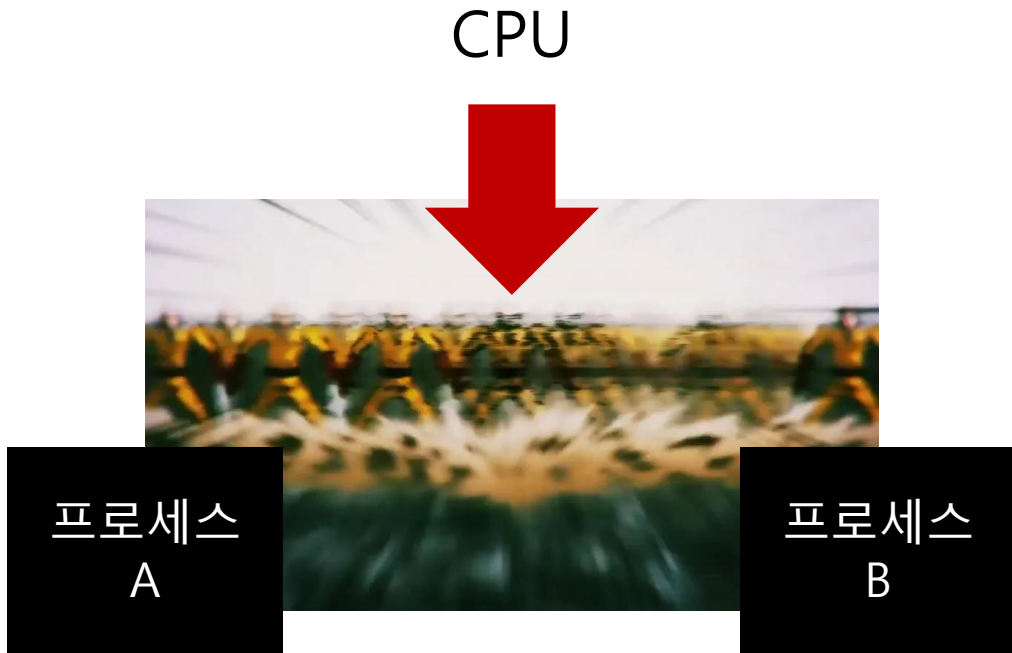


Parallelism (병렬성)

멀티 코어 환경에서 실제로 물리적으로 동시에
여러 작업을 실행하는 것



동시성 Concurrency



Concurrency (동시성)

단일 코어에서 여러 작업을 아주 빠른 속도로
번갈아가며 실행하여, 마치 동시에 진행되는 것처럼 보이게 하는 것

코어: ALU + Register + Control Unit
+ L1, L2 Cache 묶음



→ 근데 동시성 어떻게 구현함?

The background is a solid dark blue with abstract geometric shapes in lighter shades of blue. A large, light blue curved shape is on the right side, and a dark blue diagonal band runs from the bottom left towards the top right.

03

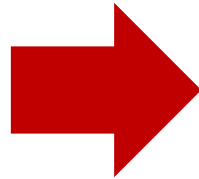
Context Switching

Context Switching

03 Context Switching

동시성 구현의 문제

```
Thread A  
  
int x = 10;  
x += 20;  
x *= 3;  
  ↑  
  여기까지 실행함
```



스레드 B를 실행하려면?

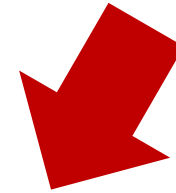
- 스레드 A의 실행 상태 덮어쓰움
- 나중에 스레드 A로 돌아오면
어디까지 진행했는지 정보가 없음
- A의 진행 상태를 어딘가에
저장해야함

03 Context Switching

Context Switching

프로세스의 저장 공간

정의: CPU가 실행 중인 프로세스(또는 스레드)를 바꾸기 위해 현재 상태를 저장하고 다음 프로세스의 상태를 복원하는 과정

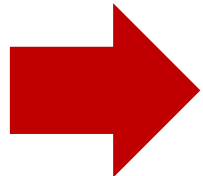
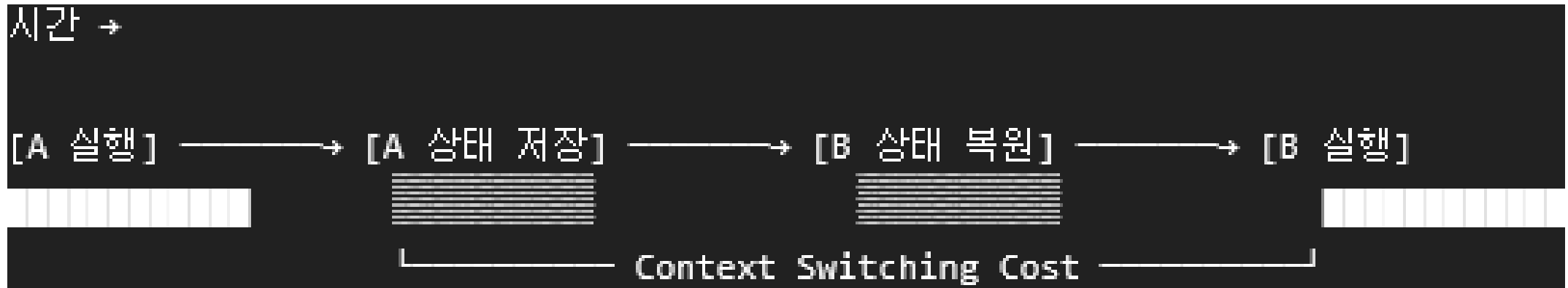


과정: Process A 실행 중단 → A의 Context(레지스터, PC 등)를 PCB에 백업 → 운영체제 스케줄러 작동 → Process B의 PCB에서 Context 복원 → Process B 실행

```
Thread A 실행
  ↓
[A의 CPU 실행 상태 저장]
  ↓
[B의 CPU 실행 상태 복원]
  ↓
Thread B 실행
```

03 Context Switching

Context Switching 의 비용



스레드 너무 늘리면 오히려 성능 안좋아짐